

ABSTRACT

This project presents a simple animation program developed using the C programming language. The program enables users to select and animate ASCII representations of various vehicles, such as cars, trucks, bikes, buses, and trains. By utilizing fundamental animation techniques, including frame-by-frame movement and smooth transitions, this project provides an interactive and engaging user experience. The implementation follows a modular approach, incorporating separate modules for menu display, user input handling, vehicle animation, and error management. The primary objective of this project is to demonstrate the basic principles of animation using console-based graphics in C. It serves as an introductory step for beginners in programming and animation, helping them understand key concepts such as loops, functions, switch-case structures, and system-level commands for screen manipulation. The project also emphasizes structured programming practices, making it an excellent learning resource for students and hobbyists. This animation system can be further extended with additional features such as user-defined vehicle designs, real-time speed control, background effects, and multi-object animations. Despite its simplicity, the project lays the foundation for more advanced applications in game development, simulations, and interactive graphical interface.

INRODUCTION

Animation is a powerful tool for storytelling, education, and entertainment. With the rise of digital technology, animation has become an integral part of various industries, including film, television, gaming, and advertising. Creating animations can be a complex task, requiring significant expertise and resources. However, with the help of programming languages, it is possible to create simple animations that can be used for various purposes. Animation programming is a fascinating field that combines art, design, and programming to create engaging and interactive experiences. With the increasing demand for animated content in various industries, including film, television, gaming, and advertising, the need for skilled animation programmers has never been greater. Creating animations can be a time-consuming process, especially for complex animations. Some animation software may have limited customization options, making it difficult to create unique animations. Animations may not be compatible with all devices or browsers, leading to playback issues.

MODULES OF THE PROJECT

Module 1: Display Menu Module

- **Description:** Displays a menu of available vehicle options to the user. Provides a simple interface for selection. Uses printf statements to print numbered options for different vehicles. Prompts the user to enter a choice. Could be modified to handle user errors, such as entering non-numeric values. Could add a feature to return to the menu if the user wants to select another vehicle.

Module 2: Vehicle Animation Module

- **Description:** Animates the selected vehicle across the screen using a loop. Uses a for loop to move the vehicle incrementally. system(CLEAR_SCREEN) clears the console screen. Prints the vehicle at each step, introducing a delay using usleep(100000). Could add user-defined speed control for animation. Could improve smoothness by using a different screen clearing method.

Module 3: User Input Module

- **Description:** Manages user input and triggers the correct animation based on the user's selection. Calls displayMenu(). Reads the user's input using scanf("%d", &choice). Uses a switch statement to call the appropriate animation function. If an invalid choice is entered, it prints an error message and exits. Could validate input before calling the animation function. Could add an option to exit gracefully or restart the menu instead of exiting immediately.

Module 4: Vehicle Data Module

- **Description:** Stores ASCII representations of vehicles in 2D character arrays. It Could allow users to create custom vehicles by inputting their own ASCII art. It Could store vehicles in a separate file for better modularity.

Module 5: Animation Control Module

- **Description:** Manages animation logic, similar to Module 2. It Could allow dynamic resizing for different screen sizes. Could add background effects or multiple moving objects.

Module 6: Error Handling Module

- **Description:** Handles invalid user input. If the user enters an invalid number, prints "Invalid choice! Exiting..." and exits the program. Instead of exiting immediately, it could prompt the user to re-enter a valid choice. Could provide feedback on expected inputs.

SYSTEM ANALYSIS

EXISTING SYSTEM

The existing system for creating simple animations typically involves using a combination of software tools and programming languages. Here are some of the key components :

- **Planning and Design:** Create a concept, script, and storyboard for the animation.
- **Asset Creation:** Design and create visual assets, such as characters, backgrounds, and props.
- **Animation Creation:** Use animation software or programming languages to create the animation.
- **Testing and Debugging:** Test the animation for errors and bugs, and debug as needed.
- **Deployment:** Deploy the animation on a website, social media, or other platform.

Limitations of the Existing System

- **Step Learning Curve:** Animation software and programming languages can be complex and difficult to learn.
- **Time-Consuming:** Creating animations can be a time-consuming process, especially for complex animations.
- **Limited Customization:** Some animation software may have limited customization options, making it difficult to create unique animations.
- **Compatibility Issues:** Animations may not be compatible with all devices or browsers, leading to playback issues.

PROPOSED SYSTEM

This system is a simple text-based vehicle animation program written in C. It allows users to choose from various vehicle options (Car, Truck, Bike, Bus, and Train) and displays an animated movement of the selected vehicle across the console screen. The animation is achieved by continuously updating the screen while shifting the ASCII representation of the vehicle from left to right.

- **System Overview:** The program provides a menu-driven interface where the user selects a vehicle type. After selection, the vehicle is displayed moving across the screen in a loop using timed delays to create a smooth animation effect.
- **Objectives:** To create a simple console-based animation system using C. To demonstrate basic ASCII animation techniques. To provide an interactive and engaging experience using text-based graphics. To utilize system functions like `system()` for clearing the screen and `usleep()` for animation timing.
- **Features of the System:** In Menu-driven Selection Users can select a vehicle from the provided list. ASCII Art Animation: Vehicles are represented using ASCII characters and animated by shifting them across the screen. Cross-Platform Compatibility: The program supports both Windows (`cls` command) and Linux/macOS (`clear` command) for clearing the screen. Smooth Movement: Uses `usleep(100000)` to create a smooth movement effect.

SYSTEM REQUIREMENTS

1.HARDWARE REQUIREMENTS

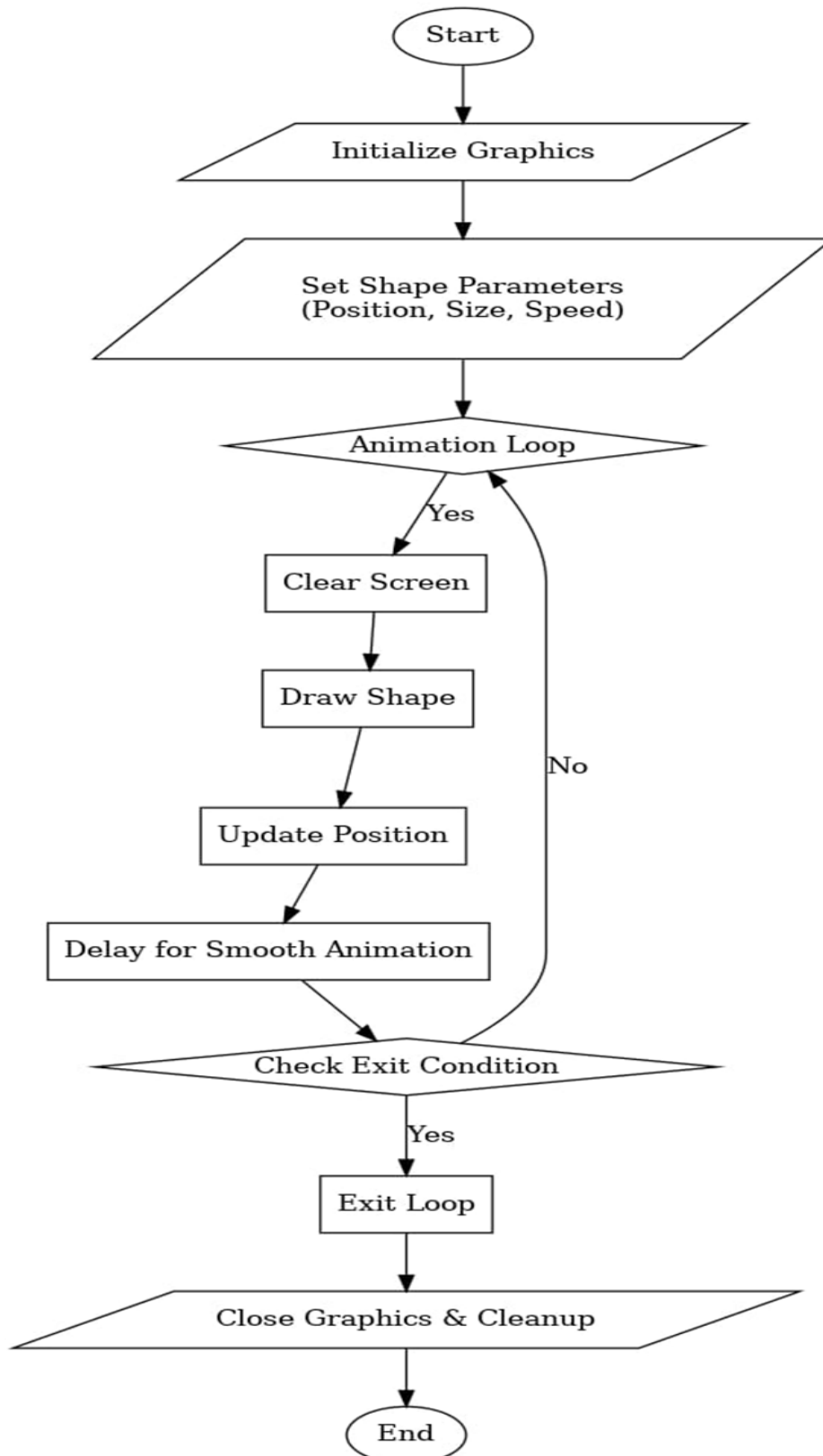
- Computer or mobile device with a compatible operating system.
- **CPU:** Intel Core i3(newer).
- **Display:** Minimum 1366×768 resolution, better with Full HD (1920×1080).
- **Battery life:** At least 3-4 hours to work without interruptions.
- Sufficient RAM and storage for running animation software

2. SOFTWARE REQUIREMENTS

- Animation software or programming language, such as C programming language
- Operating System: Windows (Version 4.18.25010.11)
- Code Editor/IDE: DEV C++

SYSTEM DESIGN

FLOW CHART



SYSTEM IMPLEMENTATION

PROJECT MODULE CODE

- **Module 1: Display Menu Module**

- **Description:** This module displays the main menu to the user, allowing them to select a vehicle to animate.

CODE:

```
void displayMenu()
{
    printf("Select a vehicle to animate:\n");
    printf("1. Car\n");
    printf("2. Truck\n");
    printf("3. Bike\n");
    printf("4. Bus\n");
    printf("5. Train\n");
    printf("Enter your choice: ");
}
```

- **Module 2: Vehicle Animation Module**

- **Description:** This module animates the selected vehicle across the screen by printing the vehicle's shape and introducing a delay.

CODE:

```
void animateVehicle(char vehicle[][40], int rows)
{
    int i, j;
    for (i = 0; i < 40; i++)
    {
        system(CLEAR_SCREEN);
        int row;
        for (row = 0; row < rows; row++)
        {
            for (j = 0; j < i; j++)
            {
                printf(" ");
            }
        }
    }
}
```

```

        printf("%s\n", vehicle[row]);
    }
    usleep(100000);
}
}

```

- **Module 3: User Input Module**

- **Description:** This module gets the user's choice of vehicle and calls the `animateVehicle()` function accordingly.

CODE:

```

int main()
{
    int choice;

    displayMenu();

    scanf("%d", &choice);

    printf("Starting animation...\n");

    sleep(2);

    switch (choice)
    {
        case 1:
            animateVehicle(car, 4);

            break;

        case 2:
            animateVehicle(truck, 4);

            break;

        case 3:
            animateVehicle(bike, 4);

            break;
    }
}

```

```

        case 4:

            animateVehicle(bus, 4);

            break;

        case 5:

            animateVehicle(train, 4);

            break;

        default:

            printf("Invalid choice! Exiting...\n");

            return 1;

    }

    printf("\nAnimation Complete!\n");

    return 0;

}

```

- **Module 4: Vehicle Data Module**

- **Description:** This module defines the 2D arrays representing the different vehicles (car, truck, bike, bus, train).

CODE:

```

char car[][40] =

{

    " _____",

    " _/  |\\",

    " | _ _ -'",

    "' (o) ----- (o) ' "

};

```

```

char truck[][40] =
{
    " _____",
    " ____||\\\\_",
    " |[]|",
    "(o) _____ (o)"
};

```

```

char bike[][40] =
{
    " ,o ",
    " -o-/ ",
    " / \\\ ",
    " (o)----- (o) "
};

```

```

char bus[][40] =
{
    " _____",
    " | [] [] [] [] []|",
    " o o"
};

```

```

char train[][40] =
{

```

```

" [=====" ,
" [===== (O) =====] ",
" | TRAIN | ",
" (o) ----- (o) "

};

```

- **Module 5: Animation Control Module**

- **Description:** This module controls the animation by clearing the screen, printing the vehicle, and introducing a delay.

CODE:

```

void animateVehicle(char vehicle[][40], int rows)
{
    int i, j;
    for (i = 0; i < 40; i++)
    {

        system(CLEAR_SCREEN);
        int row;
        for (row = 0; row < rows; row++)
        {
            for (j = 0; j < i; j++)
            {
                printf(" ");
            }
            printf("%s\n", vehicle[row]);
        }
        usleep(100000);
    }
}

```

- **Module 6: Error Handling Module**

- **Description:** This module handles invalid user input by printing an error message and exiting the program.

CODE:

```
default:  
printf("Invalid choice! Exiting...\n");  
return 1;
```

SOURCE CODE:

```
#include <stdio.h>

#include <stdlib.h>

#include <unistd.h> // For sleep()


#ifdef _WIN32

    #include <conio.h>

    #define CLEAR_SCREEN "cls"

#else

    #include <unistd.h>

    #define CLEAR_SCREEN "clear"

#endif


void animateVehicle(char vehicle[][40], int rows) {

    int i, j;


    for (i = 0; i < 40; i++) { // Move vehicle across the screen

        system(CLEAR_SCREEN);


        int row, j;

        for (row = 0; row < rows; row++) {

            for (j = 0; j < i; j++) {

                printf(" "); // Move the vehicle to the right

            }

        }

    }

}
```

```

        printf("%s\n", vehicle[row]); // Print vehicle row
    by row
    }

    usleep(100000); // Delay for smooth animation
}
}

void displayMenu() {
    printf("Select a vehicle to animate:\n");
    printf("1. Car\n");
    printf("2. Truck\n");
    printf("3. Bike\n");
    printf("4. Bus\n");
    printf("5. Train\n");
    printf("Enter your choice: ");
}

int main() {
    int choice;

    char car[][40] = {
        "      _____",
        "  _/   |\\\",
        " | _   _ -'",
        "' (o) ----- (o) '"
    };
};

```

```

char truck[][40] = {
    "          _____",
    " _____||\\_ ",
    " |[]| ",
    " (o) _____ (o) "
};

char bike[][40] = {
    "          ,o ",
    "      -o- /   ",
    " /           \\ ",
    " (o) ----- (o) "
};

char bus[][40] = {
    "          _____",
    " | []  []  []  []  [] |",
    " || ",
    "      O              O"
};

char train[][40] = {
    "          [=====] ",
    " [===== (O) =====] ",
    " |      TRAIN      | ",
    " (o) ----- (o) "
};

displayMenu();

```

```

scanf("%d", &choice);

printf("Starting animation...\n");
sleep(2); // Delay before animation starts

switch (choice) {
    case 1:
        animateVehicle(car, 4);
        break;
    case 2:
        animateVehicle(truck, 4);
        break;
    case 3:
        animateVehicle(bike, 4);
        break;
    case 4:
        animateVehicle(bus, 4);
        break;
    case 5:
        animateVehicle(train, 4);
        break;
    default:
        printf("Invalid choice! Exiting...\n");
        return 1;
}

printf("\nAnimation Complete!\n");
return 0;
}

```

TESTING

Software Testing is the process of evaluating a software application to ensure it functions correctly, meets user requirements, and is free of defects. It helps identify bugs, performance issues, and security vulnerabilities before deployment.

WHITE BOX TESTING

1. Unit Testing (Module-Wise Testing)

- Display Menu Module: Ensure the menu displays correctly.
- User Input Module: Validate that the correct animation is triggered for valid inputs and that errors are handled for invalid inputs.
- Vehicle Animation Module: Check if the vehicle moves smoothly and that delays are properly handled.
- Error Handling Module: Verify that appropriate error messages are displayed for incorrect inputs.

2. Control Flow Testing

- Test whether the switch (choice) statement correctly calls the respective animation function.
- Ensure the program returns to the menu or exits as expected.

3. Loop Testing

- Verify that the for loop in `animateVehicle()` runs correctly, moving the vehicle from position 0 to 39.

BLACK BOX TESTING

- Valid inputs such as selecting "1" for the car should correctly start the car animation.
- Invalid inputs like entering a letter instead of a number should display an error message.
- Boundary cases such as entering "0" or "6" should be handled with an appropriate error response.
- Rapid selection of options, such as choosing "1" and then immediately "3", should correctly execute both animations without crashes.
- Pressing Enter without input should trigger an error message instead of crashing the program.
- These tests ensure that both the internal structure and the functional behavior of the animation program are correct and robust.

OUTPUT

```
C:\Users\laksha\OneDrive\Des x + v
Select a vehicle to animate:
1. Car
2. Truck
3. Bike
4. Bus
5. Train
Enter your choice: |
```

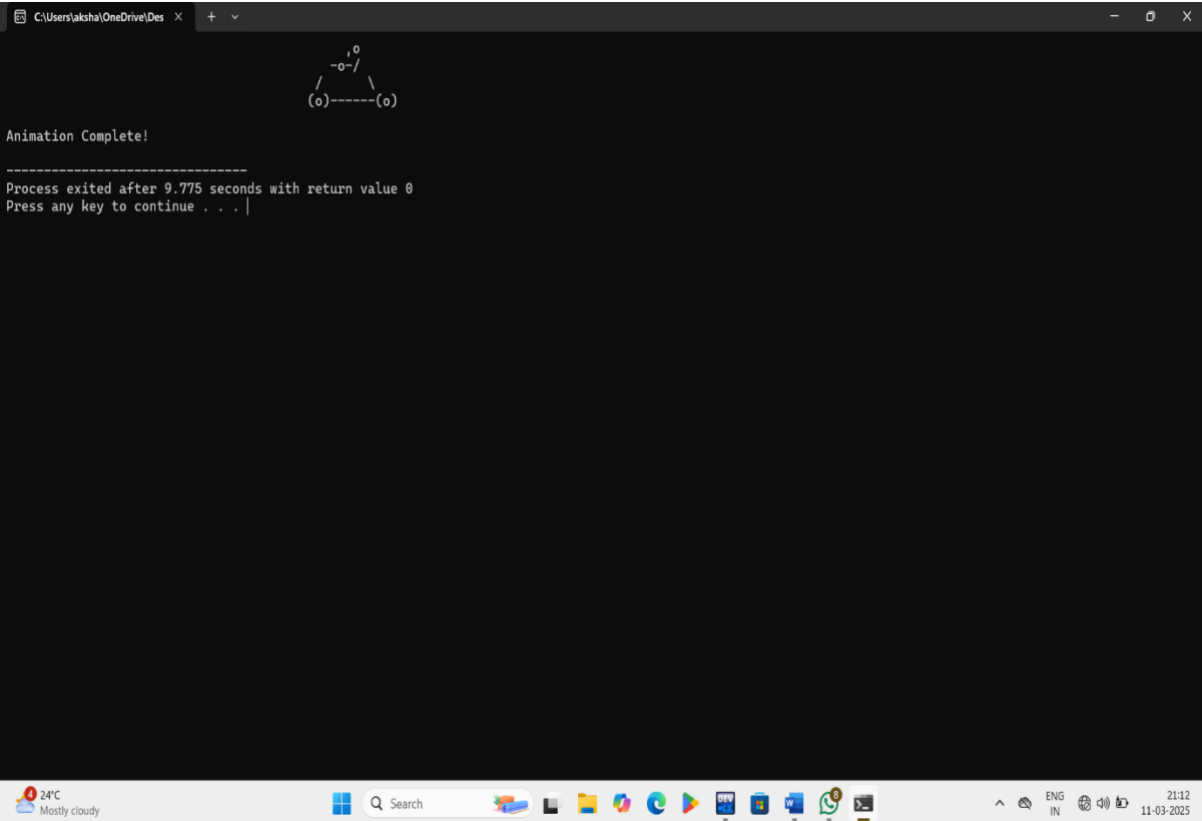
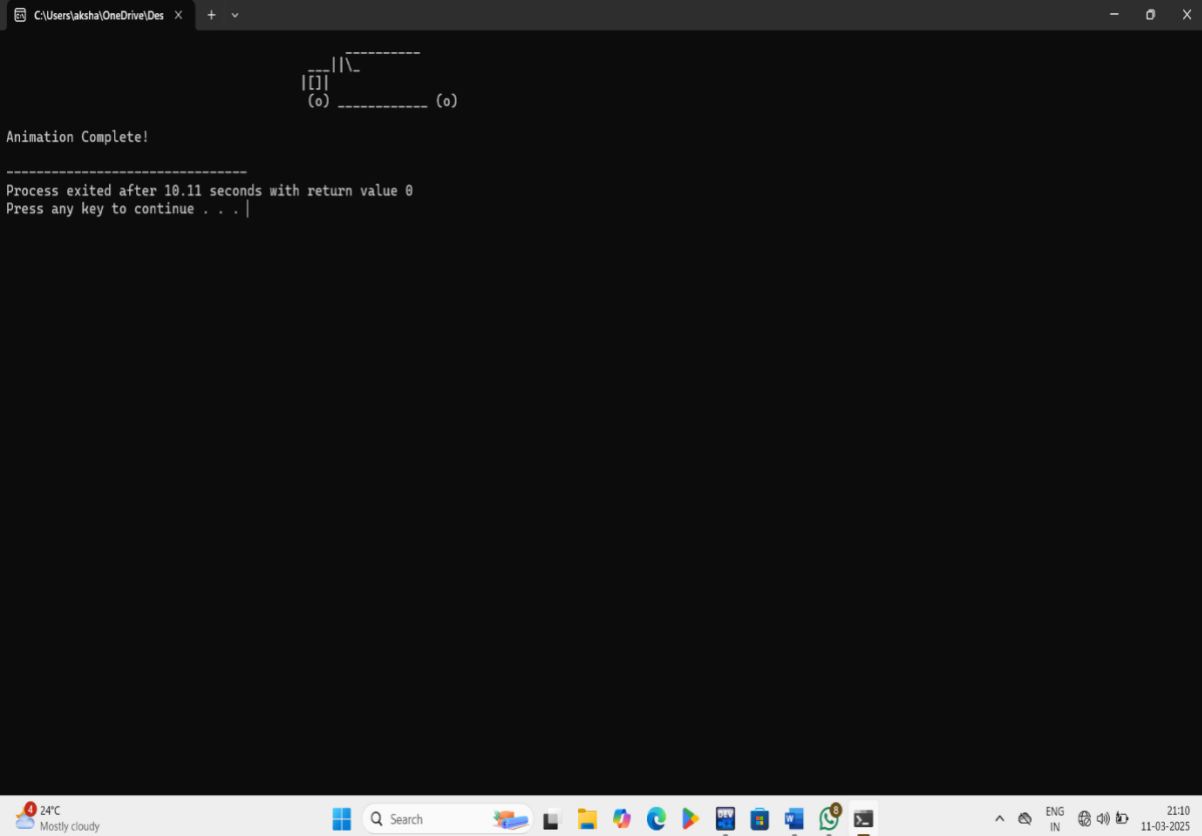
```
C:\Users\laksha\OneDrive\Des x + v
      _/  \_
     |  _  |
    '(o)------(o)'
```

Animation Complete!

Process exited after 9.1 seconds with return value 0

Press any key to continue . . . |

24°C Mostly cloudy 21:11 11-03-2025



```
C:\Users\aksha\OneDrive\Des  x  +  v  -  o  x

|  |  |  |  |
||  0      0

Animation Complete!

-----
Process exited after 10.79 seconds with return value 0
Press any key to continue . . . |
```

24°C
Mostly cloudy

Search

ENG IN 21:12 11-03-2025

```
C:\Users\aksha\OneDrive\Des  x  +  v  -  o  x

[=====]
[===== (0) =====]
|      TRAIN      |
(o)----- (o)

Animation Complete!

-----
Process exited after 10.22 seconds with return value 0
Press any key to continue . . . |
```

24°C
Mostly cloudy

Search

ENG IN 21:13 11-03-2025

CONCLUSION

This simple animation program successfully demonstrates fundamental animation concepts using C programming. Through modular design, it allows users to select and animate different ASCII vehicles, providing an interactive experience. The structured approach, including menu-driven selection, user input handling, animation logic, and error management, ensures smooth execution. This project serves as a foundational step for beginners in animation programming and can be enhanced with additional features like user-defined animations and real-time controls. Through this project, students and beginners gain hands-on experience in working with loops, functions, switch-case statements, and console output manipulation, reinforcing their understanding of core C programming concepts. Even though this project is a simple demonstration, the underlying principles of animation logic can be applied in game development, simulation software, and educational tools, making it a valuable learning experience.

BIBLIOGRAPHY/REFERENCES

1. Websites:

- GeeksforGeeks: <https://www.geeksforgeeks.org/c-programming-language/>
- TutorialsPoint: <https://www.tutorialspoint.com/cprogramming/index.htm>

2. Online Tutorials & Forums:

- Cplusplus.com (for C syntax and functions): <https://www.cplusplus.com/>
- Stack Overflow (for troubleshooting and discussions): <https://stackoverflow.com/>

3. Basic C Animation:

- Simple Console Animations in C: <https://www.geeksforgeeks.org/animation-in-c/>
- ASCII Art and Animation: <https://asciiflow.com/> (For creating ASCII vehicle designs).